

+ 26

You Shall Not Pass*!

DANIEL PFEIFER



20
26



Tribute to C++ *Seasoning: 3 Goals for Better Code*

Goal 1: No Raw Loops

What is a Raw Loop?

- A *raw loop* is any loop inside a function where the function serves purpose larger than the algorithm implemented by the loop



Goal 2: No Raw Synchronization Primitives

What are raw synchronization primitives?

- Synchronization primitives are basic constructs such as:
 - Mutex
 - Atomic
 - Semaphore
 - Memory Fence



What are **Raw** Synchronization Primitives?

A raw *loop*
is any *loop*
inside a *function*
where the *function*
serves purpose larger than
the algorithm implemented by the loop

A raw *synchronization primitive*
is any *synchronization primitive*
inside a *function*
where the *function*
serves purpose larger than
the *algorithm implemented by the loop*

A raw *synchronization primitive*
is any *synchronization primitive*
inside a *function*
where the *function*
serves purpose larger than
synchronizing the access of the data it manages

A raw *synchronization primitive*
is any *synchronization primitive*
inside a *class / function*
where the *class / function*
serves purpose larger than
synchronizing the access of the data it manages

Goal 2: No Raw Synchronization Primitives

- Use Structured Concurrency
- C++26 Execution control library

Goal 3: No Raw Pointers

What is a Raw Pointer?

- A pointer to an object with implied ownership and reference semantics
 - $T^* p = \text{new } T$
 - `unique_ptr<T>`
 - `shared_ptr<T>`



A raw *loop*
is any *loop*
inside a *function*
where the *function*
serves purpose larger than
the algorithm implemented by the loop

A raw *pointer*
is any *pointer*
inside a *function*
where the *function*
serves purpose larger than
the algorithm implemented by the loop

A raw *pointer*
is any *pointer*
inside a *function*
where the *function*
serves purpose larger than
managing the data pointed to

A raw *pointer*
is any *pointer*
inside a *class*
where the *class*
serves purpose larger than
managing the data pointed to

C++ Core Guidelines

I.11: Never transfer ownership by a raw pointer (`T*`) or reference (`T&`)

- **Alternative:** Pass ownership using a "smart pointer", such as `unique_ptr` (for exclusive ownership) and `shared_ptr` (for shared ownership). However, ...

Naming conflict!

C++ Core Guidelines

- `T*` is a raw pointer
- smart pointers are not

C++ Seasoning

- smart pointers are raw pointers
- `T*` is not necessarily a raw pointer
(it may not hold ownership)

C++ Core Guidelines

F.7: For general use, take `T*` or `T&` arguments rather than smart pointers

Reason Passing a smart pointer transfers or shares ownership and should only be used when ownership semantics are intended. A function that does not manipulate lifetime should take raw pointers or references instead.

Passing by smart pointer restricts the use of a function to callers that use smart pointers. A function that needs a widget should be able to accept any widget object, not just ones whose lifetimes are managed by a particular kind of smart pointer.

C++ Core Guidelines

- `T*` is a raw pointer
- smart pointers are not

C++ Seasoning

- smart pointers are raw pointers
- `T*` is not necessarily a raw pointer
(it may not hold ownership)

We need a better Goal

Why Pointers

Why Pointers

- details unknown
 - runtime dynamic count (container)
 - runtime dynamic type (polymorphism)

Why Pointers

- details unknown
 - runtime dynamic count (container)
 - runtime dynamic type (polymorphism)
- details private
 - API/ABI stability
 - faster compilation (compiler firewall)

Why Pointers

- details unknown
 - runtime dynamic count (container)
 - runtime dynamic type (polymorphism)
- details private
 - API/ABI stability
 - faster compilation (compiler firewall)
- complexity / stability requirement
 - $O(1)$ copy (views)
 - stability (list vs vector)
 - Deferral of rare and expensive work (copy-on-write)

```
Type* object = ...
```

- Am I responsible for delete / free ?
- What actions will cause this to dangle?
- Am I allowed to modify?
- What actions may cause a modification?
- Does modification require synchronization?
- Does reading require synchronization?

```
Type const* object = ...
```

- ~~Am I responsible for `delete` / `free` ?~~
- What actions will cause this to dangle?
- ~~Am I allowed to modify?~~
- What actions may cause a modification?
- ~~Does modification require synchronization?~~
- Does reading require synchronization?

```
std::shared_ptr<Type> object = ...
```

- ~~Am I responsible for `delete` / `free` ?~~
- ~~What actions will cause this to dangle?~~
- Am I allowed to modify?
- What actions may cause a modification?
- Does modification require synchronization?
- Does reading require synchronization?

```
std::shared_ptr<Type const> object = ...
```

- ~~Am I responsible for `delete` / `free` ?~~
- ~~What actions will cause this to dangle?~~
- ~~Am I allowed to modify?~~
- What actions may cause a modification?
- ~~Does modification require synchronization?~~
- Does reading require synchronization?

```
std::unique_ptr<Type> object = ...
```

- ~~Am I responsible for `delete` / `free` ?~~
- ~~What actions will cause this to dangle?~~
- Am I allowed to modify?
- What actions may cause a modification?
- Does modification require synchronization?
- Does reading require synchronization?

```
int object = ...
```

- Am I responsible for `delete` / `free` ?
- What actions will cause this to dangle?
- Am I allowed to modify?
- What actions may cause a modification?
- Does modification require synchronization?
- Does reading require synchronization?

```
Class object = ...
```

- ~~Am I responsible for `delete` / `free` ?~~
- ~~What actions will cause this to dangle?~~
- Am I allowed to modify?
- What actions may cause a modification?
- Does modification require synchronization?
- Does reading require synchronization?

Does `class` behave like `int`, or `Type*`?

There is a single place to look at!

Local Reasoning

Require that a pointer be the only data member of a class?

Require that a pointer be the only data member of a class?

No.

Require that a pointer be the only data member of a class?

No.

Example: A node class in a doubly linked list holds multiple pointers.
Serves a single purpose. Allows local reasoning.

Only allow pointers as class data members?

Only allow pointers as class data members?

No.

Only allow pointers as class data members?

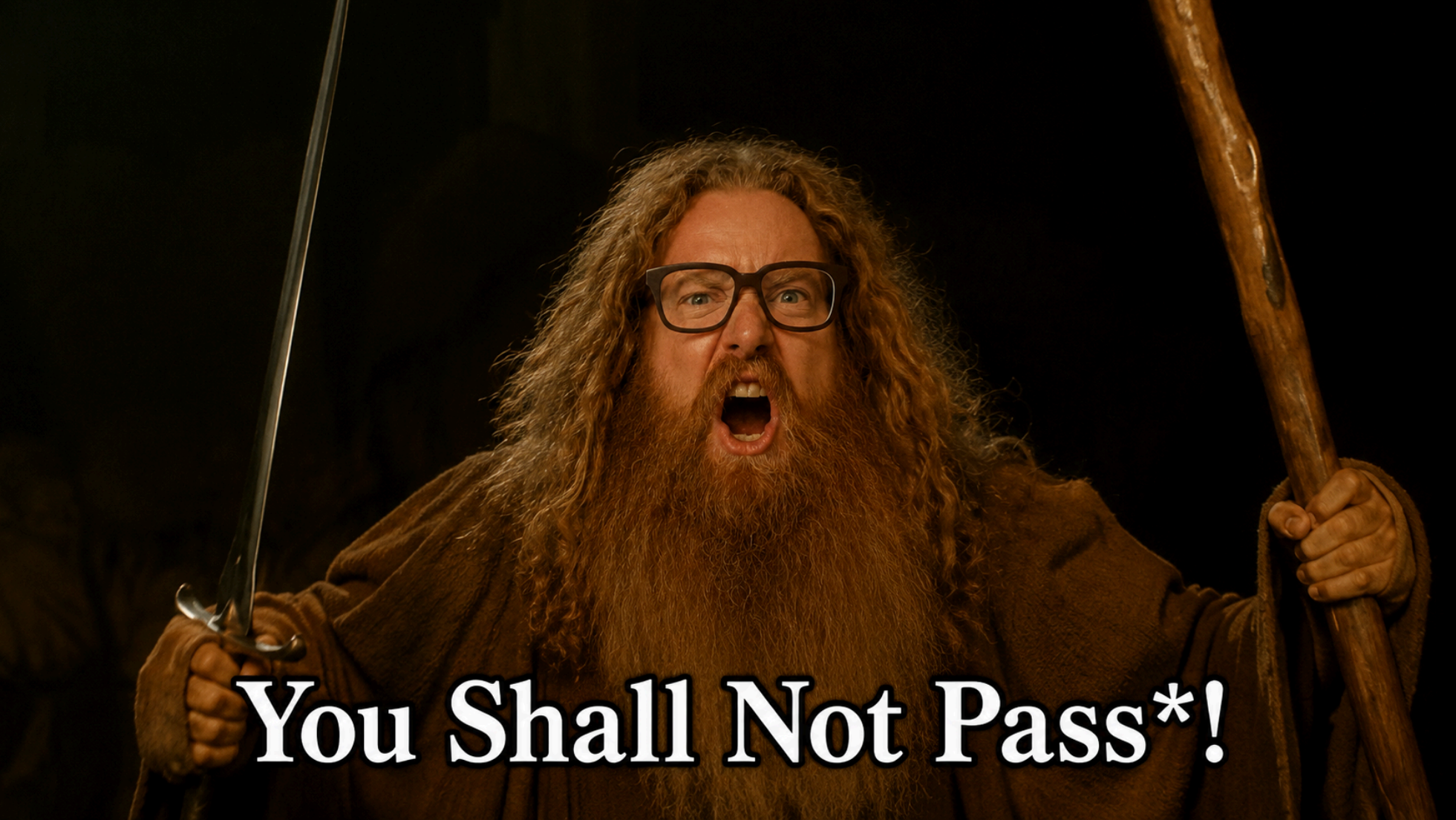
No.

A pointer that is local to a function still allows local reasoning.

The ability for local reasoning ends as soon as a pointer is passed around.

The ability for local reasoning ends as soon as a pointer is passed around.

Either as return type, or as function parameter.

A man with a long, flowing brown beard and thick-rimmed black glasses is shown from the chest up. He is wearing a brown, textured robe. He holds a sword in his right hand, with the blade pointing upwards, and a wooden staff in his left hand. His mouth is wide open in a shout, and his eyes are wide. The background is dark and indistinct.

You Shall Not Pass*!

The End

Bonus Slides

Have you seen this pattern before?

```
class Foo
{
public:
    using Ptr = std::shared_ptr<Foo>;

    Ptr Make();

private:
    Foo();
};
```

```
{
    // This is prevented:
    // Foo* foo = new Foo;

    // This is allowed
    Foo::Ptr foo = Foo::Make();
}
```

Start from scratch.

RAII: Resource Acquisition Is Initialization

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

private:
    FooImpl* _impl;
};
```

```
{
    // Acquire a resource by initialization
    Foo foo;
}
```

RAII: Resource Acquisition Is Initialization

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

private:
    FooImpl* _impl;
};
```

```
{
    // Acquire a resource by initialization
    Foo foo;

    // Release at destruction
}
```

RAII: Resource Acquisition Is Initialization

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

private:
    FooImpl* _impl;
};
```

```
{
    // Acquire a resource by initialization
    Foo foo;

    if (...) {
        return;
    }

    // Release at destruction
}
```

RAII: Resource Acquisition Is Initialization

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

private:
    FooImpl* _impl;
};
```

```
{
    // Acquire a resource by initialization
    Foo foo;

    if (...) {
        return;
    }

    if (...) {
        throw Error;
    }

    // Release at destruction
}
```

RAII: Resource Acquisition Is Initialization

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

private:
    FooImpl* _impl;
};
```

```
{
    // Acquire a resource by initialization
    Foo foo;

    if (...) {
        return;
    }

    if (...) {
        throw Error;
    }

    // Release at destruction
}
```

RAII provides *basic exception safety* guarantee.

Trick question: Can you see the bug?

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

private:
    FooImpl* _impl;
};
```

A bug is a **defect** that may lead to a **fault**.

Trick question: Can you see the *defect*?

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

private:
    FooImpl* _impl;
};
```

```
{
    Foo a;
}
```

Trick question: Can you see the *defect*?

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

private:
    FooImpl* _impl;
};
```

```
{
    Foo a;
    Foo b = a;
}
```

Trick question: Can you see the *defect*?

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

private:
    FooImpl* _impl;
};
```

```
{
    Foo a;
    Foo b = a;
    // fault!
}
```

Trick question: Can you see the *defect*?

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

private:
    FooImpl* _impl;
};
```

```
{
    Foo a;
    Foo b = a;
    // fault!
}
```

- The user code is correct.

Trick question: Can you see the *defect*?

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

private:
    FooImpl* _impl;
};
```

```
{
    Foo a;
    Foo b = a;
    // fault!
}
```

- The user code is correct.
- The destructor is correct.

Trick question: Can you see the *defect*?

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        _impl = other._impl;
    }

    Foo& operator=(Foo const& other) {
        _impl = other._impl;
        return *this;
    }

private:
    FooImpl* _impl;
};
```

```
{
    Foo a;
    Foo b = a;
    // fault!
}
```

- The user code is correct.
- The destructor is correct.
- The defect is provided by the compiler!

Implementing copy

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        _impl = other._impl;
    }

    Foo& operator=(Foo const& other) {
        _impl = other._impl;
        return *this;
    }

private:
    FooImpl* _impl;
};
```

```
{
    Foo a;
    Foo b = a;
    // fault!
}
```

Copy constructor causes fault!

Implementing copy

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        _impl = other._impl;
        return *this;
    }

private:
    FooImpl* _impl;
};
```

```
{
    Foo a;
    Foo b = a;
}
```

Copy constructor ok

Implementing copy

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        _impl = other._impl;
        return *this;
    }

private:
    FooImpl* _impl;
};
```

```
{
    Foo a;
    Foo b = a;
    a = b;
    // fault!
}
```

Assignment causes fault!

Implementing copy

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        _impl = new FooImpl(*other._impl);
        return *this;
    }

private:
    FooImpl* _impl;
};
```

```
{
    Foo a;
    Foo b = a;
    a = b; // memory leak!
}
```

Assignment causes leak!

Implementing copy

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        delete _impl;
        _impl = new FooImpl(*other._impl);
        return *this;
    }

private:
    FooImpl* _impl;
};
```

```
{
    Foo a;
    Foo b = a;
    a = b;
}
```

Assignment leak is fixed, but ...

Implementing copy

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        delete _impl;
        _impl = new FooImpl(*other._impl);
        return *this;
    }

private:
    FooImpl* _impl;
};
```

```
{
    Foo a;
    Foo b = a;
    a = b;
    a = a; // fault!
}
```

Assignment leak is fixed, but
self assignment causes fault

Implementing copy

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        if (this != &other) {
            delete _impl;
            _impl = new FooImpl(*other._impl);
        }
        return *this;
    }

private:
    FooImpl* _impl;
};
```

```
{
    Foo a;
    Foo b = a;
    a = b;
    a = a;
}
```

Assignment is not exception safe

Implementing copy

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        if (this != &other) {
            FooImpl* tmp = new FooImpl(*other._impl);
            delete _impl;
            _impl = tmp;
        }
        return *this;
    }

private:
    FooImpl* _impl;
};
```

```
{
    Foo a;
    Foo b = a;
    a = b;
    a = a;
}
```

Assignment is optimized for edge case

Implementing copy

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        FooImpl* tmp = new FooImpl(*other._impl);
        delete _impl;
        _impl = tmp;
        return *this;
    }

private:
    FooImpl* _impl;
};
```

```
{
    Foo a;
    Foo b = a;
    a = b;
    a = a;
}
```

Assignment ok

Rule of three (C++98 Guideline)

If a class requires a user-defined *destructor*, a user-defined *copy constructor*, or a user-defined *copy assignment operator*, it almost certainly requires *all three*.

Templates, beware!

```
template <typename T>
class Foo
{
public:
    Foo() { _impl = new FooImpl<T>; }
    ~Foo() { delete _impl; }

    template <typename U>
    Foo(Foo<U> const& other) {
        _impl = new FooImpl<T>(*other._impl);
    }

    template <typename U>
    Foo& operator=(Foo<U> const& other) { ... }

private:
    FooImpl<T>* _impl;
};
```

```
{
    Foo<float> a;
    Foo<double> b = a;
    Foo<float> c = a;
    // fault!
}
```

A constructor template or assignment operator template does not inhibit the provision of the copy constructor or copy assignment operator.

C++11

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        FooImpl* tmp = new FooImpl(*other._impl);
        delete _impl;
        _impl = tmp;
        return *this;
    }

private:
    FooImpl* _impl;
};
```

C++11

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        FooImpl* tmp = new FooImpl(*other._impl);
        delete _impl;
        _impl = tmp;
        return *this;
    }

    Foo(Foo&& other) noexcept {
        _impl = other._impl;
        other._impl = nullptr;
    }
}
```

C++11

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        assert(other._impl);
        FooImpl* tmp = new FooImpl(*other._impl);
        delete _impl;
        _impl = tmp;
        return *this;
    }

    Foo(Foo&& other) noexcept {
        _impl = other._impl;
        other._impl = nullptr;
    }
}
```

Reasoning about value semantics

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        assert(other._impl);
        FooImpl* tmp = new FooImpl(*other._impl);
        delete _impl;
        _impl = tmp;
        return *this;
    }

    Foo(Foo&& other) noexcept {
        _impl = other._impl;
        other._impl = nullptr;
    }
}
```

```
{
    Foo a;
    Foo b = a;
    assert(b == a);
}
```

Required to reason about equality.

Reasoning about value semantics

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        assert(other._impl);
        FooImpl* tmp = new FooImpl(*other._impl);
        delete _impl;
        _impl = tmp;
        return *this;
    }

    Foo(Foo&& other) noexcept {
        _impl = other._impl;
        other._impl = nullptr;
    }
}
```

```
{
    Foo a;
    Foo b = a;
    assert(b == a);
}
```

Required to reason about equality.

Reasoning about value semantics

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        assert(other._impl);
        FooImpl* tmp = new FooImpl(*other._impl);
        delete _impl;
        _impl = tmp;
        return *this;
    }

    Foo(Foo&& other) noexcept {
        _impl = other._impl;
        other._impl = nullptr;
    }
}
```

```
{
    Foo a;
    Foo b = a;
    assert(b == a);
}
```

Required to reason about equality.

Reasoning about value semantics

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        assert(other._impl);
        FooImpl* tmp = new FooImpl(*other._impl);
        delete _impl;
        _impl = tmp;
        return *this;
    }

    Foo(Foo&& other) noexcept {
        _impl = other._impl;
        other._impl = nullptr;
    }
}
```

```
{
    Foo a;
    Foo b = a;
    assert(b == a);
}
```

Required to reason about equality.

Reasoning about value semantics

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        assert(other._impl);
        FooImpl* tmp = new FooImpl(*other._impl);
        delete _impl;
        _impl = tmp;
        return *this;
    }

    Foo(Foo&& other) noexcept {
        _impl = other._impl;
        other._impl = nullptr;
    }
}
```

```
{
    Foo a;
    Foo b = a;
    assert(b == a);
}
```

Required to reason about equality.

Status: 66 Lines of pure boilerplate. No business logic yet.

std::exchange (C++14)

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        assert(other._impl);
        FooImpl* tmp = new FooImpl(*other._impl);
        delete _impl;
        _impl = tmp;
        return *this;
    }

    Foo(Foo&& other) noexcept {
        _impl = other._impl;
        other._impl = nullptr;
    }
}
```

std::exchange (C++14)

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        assert(other._impl);
        FooImpl* tmp = new FooImpl(*other._impl);
        delete _impl;
        _impl = tmp;
        return *this;
    }

    Foo(Foo&& other) noexcept {
        _impl = std::exchange(other._impl, nullptr);
    }
}
```

std::swap (C++98)

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        assert(other._impl);
        FooImpl* tmp = new FooImpl(*other._impl);
        delete _impl;
        _impl = tmp;
        return *this;
    }

    Foo(Foo&& other) noexcept {
        _impl = std::exchange(other._impl, nullptr);
    }
}
```

std::swap (C++98)

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        Foo tmp(other);
        std::swap(_impl, tmp._impl);
        return *this;
    }

    Foo(Foo&& other) noexcept {
        _impl = std::exchange(other._impl, nullptr);
    }

    Foo& operator=(Foo&& other) noexcept {
```

std::swap (C++98)

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        Foo tmp(other);
        std::swap(_impl, tmp._impl);
        return *this;
    }

    Foo(Foo&& other) noexcept {
        _impl = std::exchange(other._impl, nullptr);
    }

    Foo& operator=(Foo&& other) noexcept {
```

std::swap (C++98)

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept {
        _impl = std::exchange(other._impl, nullptr);
    }

    Foo& operator=(Foo&& other) noexcept {
        Foo tmp(std::move(other));
        std::swap( _impl, tmp._impl);
    }
}
```

`std::unique_ptr` (C++11), `std::make_unique` (C++14)

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept {
        _impl = std::exchange(other._impl, nullptr);
    }

    Foo& operator=(Foo&& other) noexcept {
        Foo tmp(std::move(other));
        std::swap(_impl, tmp._impl);
    }
}
```

`std::unique_ptr` (C++11), `std::make_unique` (C++14)

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() { delete _impl; }

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept {
        _impl = std::exchange(other._impl, nullptr);
    }

    Foo& operator=(Foo&& other) noexcept {
        Foo tmp(std::move(other));
        std::swap(_impl, tmp._impl);
    }
}
```

`std::unique_ptr` (C++11), `std::make_unique` (C++14)

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() = default;

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept {
        _impl = std::exchange(other._impl, nullptr);
    }

    Foo& operator=(Foo&& other) noexcept {
        Foo tmp(std::move(other));
        std::swap(_impl, tmp._impl);
    }
}
```

`std::unique_ptr` (C++11), `std::make_unique` (C++14)

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() = default;

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;

    Foo& operator=(Foo&& other) noexcept {
        Foo tmp(std::move(other));
        std::swap(_impl, tmp._impl);
        return *this;
    }
}
```

`std::unique_ptr` (C++11), `std::make_unique` (C++14)

```
class Foo
{
public:
    Foo() { _impl = new FooImpl; }
    ~Foo() = default;

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = new FooImpl(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        assert(_impl && other._impl);
        return *_impl == *other._impl;
    }
}
```

`std::unique_ptr` (C++11), `std::make_unique` (C++14)

```
class Foo
{
public:
    Foo() { _impl = std::make_unique<FooImpl>(); }
    ~Foo() = default;

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = std::make_unique<FooImpl>(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        assert(_impl && other._impl);
        return *_impl == *other._impl;
    }
}
```

operator $\Leftrightarrow$ (C++20)

```
class Foo
{
public:
    Foo() { _impl = std::make_unique<FooImpl>(); }
    ~Foo() = default;

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = std::make_unique<FooImpl>(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        assert(_impl && other._impl);
        return *_impl == *other._impl;
    }
}
```

operator $\Leftrightarrow$ (C++20)

```
class Foo
{
public:
    Foo() { _impl = std::make_unique<FooImpl>(); }
    ~Foo() = default;

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = std::make_unique<FooImpl>(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        assert(_impl && other._impl);
        return *_impl == *other._impl;
    }
}
```

operator $\Leftrightarrow$ (C++20)

```
class Foo
{
public:
    Foo() { _impl = std::make_unique<FooImpl>(); }
    ~Foo() = default;

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = std::make_unique<FooImpl>(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        assert(_impl && other._impl);
        return *_impl == *other._impl;
    }
}
```

Down to 31 lines of boilerplate.

Business Logic

```
class Foo
{
public:
    Foo() { _impl = std::make_unique<FooImpl>(); }
    ~Foo() = default;

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = std::make_unique<FooImpl>(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        assert(_impl && other._impl);
        return *_impl == *other._impl;
    }
}
```

Prefer terms *observer* and *modifier* over *getter* and *setter*.

Business Logic

```
class Foo
{
public:
    Foo() { _impl = std::make_unique<FooImpl>(); }
    ~Foo() = default;

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = std::make_unique<FooImpl>(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        assert(_impl && other._impl);
        return *_impl == *other._impl;
    }
}
```

Prefer terms *observer* and *modifier* over *getter* and *setter*.

Constness Pitfall

```
class Foo
{
public:
    Foo() { _impl = std::make_unique<FooImpl>(); }
    ~Foo() = default;

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = std::make_unique<FooImpl>(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        assert(_impl && other._impl);
        return *_impl == *other._impl;
    }
}
```

```
struct FooImpl {
    std::size_t size() {
        std::println("I am mutable");
        return _size;
    }

    std::size_t size() const {
        std::println("I am const");
        return _size;
    }
};
```

Constness Pitfall

```
class Foo
{
public:
    Foo() { _impl = std::make_unique<FooImpl>(); }
    ~Foo() = default;

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = std::make_unique<FooImpl>(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        assert(_impl && other._impl);
        return *_impl == *other._impl;
    }
}
```

```
struct FooImpl {
    std::size_t size() {
        std::println("I am mutable");
        return _size;
    }

    std::size_t size() const {
        std::println("I am const");
        return _size;
    }
};
```

I am mutable

Constness Pitfall

```
class Foo
{
public:
    Foo() { _impl = std::make_unique<FooImpl>(); }
    ~Foo() = default;

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = std::make_unique<FooImpl>(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        assert(_impl && other._impl);
        return *_impl == *other._impl;
    }
}
```

```
struct FooImpl {
    std::size_t size() {
        std::println("I am mutable");
        return _size;
    }

    std::size_t size() const {
        std::println("I am const");
        return _size;
    }
};
```

I am mutable

Constness Pitfall

```
class Foo
{
public:
    Foo() { _impl = std::make_unique<FooImpl>(); }
    ~Foo() = default;

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = std::make_unique<FooImpl>(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        assert(_impl && other._impl);
        return *_impl == *other._impl;
    }
}
```

```
struct FooImpl {
    std::size_t size() {
        std::println("I am mutable");
        return _size;
    }

    std::size_t size() const {
        std::println("I am const");
        return _size;
    }
};
```

I am const

Constness Pitfall

```
class Foo
{
public:
    Foo() { _impl = std::make_unique<FooImpl>(); }
    ~Foo() = default;

    Foo(Foo const& other) {
        assert(other._impl);
        _impl = std::make_unique<FooImpl>(*other._impl);
    }

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        return impl() == other.impl();
    }
}
```

```
struct FooImpl {
    std::size_t size() {
        std::println("I am mutable");
        return _size;
    }

    std::size_t size() const {
        std::println("I am const");
        return _size;
    }
};
```

Constness Pitfall

```
class Foo
{
public:
    Foo() : _impl{std::make_unique<FooImpl>()} {}
    ~Foo() = default;

    Foo(Foo const& other)
        : _impl{std::make_unique<FooImpl>(other.impl())}
    {}

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        return impl() == other.impl();
    }

    auto operator<=>(Foo const& other) const {
```

```
struct FooImpl {
    std::size_t size() {
        std::println("I am mutable");
        return _size;
    }

    std::size_t size() const {
        std::println("I am const");
        return _size;
    }
};
```

Optimization: If the class does not have *modifiers*, consider Transparent Sharing.

Optimization: Transparent Sharing

```
class Foo
{
public:
    Foo() : _impl{std::make_unique<FooImpl>()} {}
    ~Foo() = default;

    Foo(Foo const& other)
        : _impl{std::make_unique<FooImpl>(other.impl())}
    {}

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        return impl() == other.impl();
    }

    auto operator<=>(Foo const& other) const {
```

Optimization: Transparent Sharing

```
class Foo
{
public:
    Foo() : _impl{std::make_unique<FooImpl>()} {}
    ~Foo() = default;

    Foo(Foo const& other)
        : _impl{std::make_unique<FooImpl>(other.impl())}
    {}

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        return impl() == other.impl();
    }

    auto operator<=>(Foo const& other) const {
```

Optimization: Transparent Sharing

```
class Foo
{
public:
    Foo(std::size_t s) : _impl{std::make_unique<FooImpl>(s)} {}
    ~Foo() = default;

    Foo(Foo const& other)
        : _impl{std::make_unique<FooImpl>(other.impl())}
    {}

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        return impl() == other.impl();
    }

    auto operator<=>(Foo const& other) const {
```

Optimization: Transparent Sharing

```
class Foo
{
public:
    Foo(std::size_t s) : _impl{std::make_unique<FooImpl>(s)} {}
    ~Foo() = default;

    Foo(Foo const& other)
        : _impl{std::make_unique<FooImpl>(other.impl())}
    {}

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        return impl() == other.impl();
    }

    auto operator<=>(Foo const& other) const {
```

Optimization: Transparent Sharing

```
class Foo
{
public:
    Foo(std::size_t s) : _impl{std::make_shared<FooImpl>(s)} {}
    ~Foo() = default;

    Foo(Foo const& other)
        : _impl{std::make_shared<FooImpl>(other.impl())}
    {}

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        return impl() == other.impl();
    }

    auto operator<=>(Foo const& other) const {
```

Optimization: Transparent Sharing

```
class Foo
{
public:
    Foo(std::size_t s) : _impl{std::make_shared<FooImpl>(s)} {}
    ~Foo() = default;

    Foo(Foo const& other) = default;
    Foo& operator=(Foo const& other) = default;

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        return impl() == other.impl();
    }

    auto operator<=>(Foo const& other) const {
        return impl() <=> other.impl();
    }

    bool empty() const {
        return impl().size() != 0;
    }
}
```

Optimization: Transparent Sharing

```
class Foo
{
public:
    Foo(std::size_t s) : _impl{std::make_shared<FooImpl>(s)} {}
    ~Foo() = default;

    Foo(Foo const& other) = default;
    Foo& operator=(Foo const& other) = default;

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        return _impl == other._impl || impl() == other.impl();
    }

    auto operator<=>(Foo const& other) const {
        return impl() <=> other.impl();
    }

    bool empty() const {
        return impl().size() != 0;
    }
}
```

Optimization: Transparent Sharing

```
class Foo
{
public:
    Foo(std::size_t s) : _impl{std::make_shared<FooImpl>(s)} {}
    ~Foo() = default;

    Foo(Foo const& other) = default;
    Foo& operator=(Foo const& other) = default;

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        return _impl == other._impl || impl() == other.impl();
    }

    auto operator<=>(Foo const& other) const {
        return impl() <=> other.impl();
    }

    bool empty() const {
        return impl().size() != 0;
    }
}
```

- No difference in API
- Copy has $O(1)$ complexity (atomic increment)

Rule of zero

Classes that have custom *destructors*, *copy/move constructors* or *copy/move assignment operators* should deal exclusively with ownership (which follows from the Single-responsibility principle).

Other classes should not have custom destructors, copy/move constructors or copy/move assignment operators.

std::indirect (C++26)

```
class Foo
{
public:
    Foo() : _impl{std::make_unique<FooImpl>{}} {}
    ~Foo() = default;

    Foo(Foo const& other)
        : _impl{std::make_unique<FooImpl>(other.impl())}
    {}

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        return impl() == other.impl();
    }

    auto operator<=>(Foo const& other) const {
```

- Reduce boilerplate
- Solve const propagation issue

std::indirect (C++26)

```
class Foo
{
public:
    Foo() : _impl{std::make_unique<FooImpl>()} {}
    ~Foo() = default;

    Foo(Foo const& other)
        : _impl{std::make_unique<FooImpl>(other.impl())}
    {}

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        return impl() == other.impl();
    }

    auto operator<=>(Foo const& other) const {
```

- Reduce boilerplate
- Solve const propagation issue

std::indirect (C++26)

```
class Foo
{
public:
    Foo() : _impl{std::make_unique<FooImpl>()} {}
    ~Foo() = default;

    Foo(Foo const& other)
        : _impl{std::make_unique<FooImpl>(other.impl())}
    {}

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        return impl() == other.impl();
    }

    auto operator<=>(Foo const& other) const {
```

- Reduce boilerplate
- Solve const propagation issue

std::indirect (C++26)

```
class Foo
{
public:
    Foo() : _impl{std::make_unique<FooImpl>()} {}
    ~Foo() = default;

    Foo(Foo const& other)
        : _impl{std::make_unique<FooImpl>(other.impl())}
    {}

    Foo& operator=(Foo const& other) {
        return *this = Foo(other);
    }

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        return impl() == other.impl();
    }

    auto operator<=>(Foo const& other) const {
```

- Reduce boilerplate
- Solve const propagation issue

std::indirect (C++26)

```
class Foo
{
public:
    Foo() : _impl{std::make_unique<FooImpl>{}} {}
    ~Foo() = default;

    Foo(Foo const& other) = default;
    Foo& operator=(Foo const& other) = default;

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const {
        return impl() == other.impl();
    }

    auto operator<=>(Foo const& other) const {
        return impl() <=> other.impl();
    }

    bool empty() const {
        return impl->size() != 0;
    }
};
```

- Reduce boilerplate
- Solve const propagation issue

std::indirect (C++26)

```
class Foo
{
public:
    Foo() : _impl{std::make_unique<FooImpl>{}} {}
    ~Foo() = default;

    Foo(Foo const& other) = default;
    Foo& operator=(Foo const& other) = default;

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const = default;
    auto operator<=>(Foo const& other) const = default;

    bool empty() const {
        return _impl->size() != 0;
    }

    void resize(std::size_t s) {
        _impl->resize(s);
    }
}
```

- Reduce boilerplate
- Solve const propagation issue

std::indirect (C++26)

```
class Foo
{
public:
    Foo(std::size_t s) : _impl{std::in_place, s} {}
    ~Foo() = default;

    Foo(Foo const& other) = default;
    Foo& operator=(Foo const& other) = default;

    Foo(Foo&& other) noexcept = default;
    Foo& operator=(Foo&& other) noexcept = default;

    bool operator==(Foo const& other) const = default;
    auto operator<=>(Foo const& other) const = default;

    bool empty() const {
        return _impl->size() != 0;
    }

    void resize(std::size_t s) {
        _impl->resize(s);
    }
}
```

- Reduce boilerplate
- Solve const propagation issue

std::indirect (C++26)

```
class Foo
{
public:
    Foo(std::size_t s) : _impl{std::in_place, s} {}

    bool operator==(Foo const& other) const = default;
    auto operator<=>(Foo const& other) const = default;

    bool empty() const {
        return _impl->size() != 0;
    }

    void resize(std::size_t s) {
        _impl->resize(s);
    }

private:
    std::indirect<FooImpl> _impl;
};
```

- Reduce boilerplate
- Solve const propagation issue

std::copy_on_write (P4210)

```
class Foo
{
public:
    Foo(std::size_t s) : _impl{std::in_place, s} {}

    bool operator==(Foo const& other) const = default;
    auto operator<=>(Foo const& other) const = default;

    bool empty() const {
        return _impl->size() != 0;
    }

    void resize(std::size_t s) {
        _impl->resize(s);
    }

private:
    std::indirect<FooImpl> _impl;
};
```

`std::copy_on_write` (P4210)

```
class Foo
{
public:
    Foo(std::size_t s) : _impl{std::in_place, s} {}

    bool operator==(Foo const& other) const = default;
    auto operator<=>(Foo const& other) const = default;

    bool empty() const {
        return _impl->size() != 0;
    }

    void resize(std::size_t s) {
        _impl->resize(s);
    }

private:
    std::copy_on_write<FooImpl> _impl;
};
```

- Copy has $O(1)$ complexity (atomic increment)

std::copy_on_write (P4210)

```
class Foo
{
public:
    Foo(std::size_t s) : _impl{std::in_place, s} {}

    bool operator==(Foo const& other) const = default;
    auto operator<=>(Foo const& other) const = default;

    bool empty() const {
        return _impl->size() != 0;
    }

    void resize(std::size_t s) {
        _impl.modify([&](auto& impl) { impl.resize(s); });
    }

private:
    std::copy_on_write<FooImpl> _impl;
};
```

- Copy has $O(1)$ complexity (atomic increment)
- `modify` `copies` `Impl` *iff* object is shared

std::copy_on_write (P4210)

```
class Foo
{
public:
    Foo(std::size_t s) : _impl{std::in_place, s} {}

    bool operator==(Foo const& other) const = default;
    auto operator<=>(Foo const& other) const = default;

    bool empty() const {
        return _impl->size() != 0;
    }

    void resize(std::size_t s) {
        _impl.modify([&](auto& impl) { impl.resize(s); });
    }

    void push_front(int val) {
        _impl.modify(
            [&](std::vector<int>& impl) {
                impl.push_front(val);
            }
        );
    }
};
```

- Copy has $O(1)$ complexity (atomic increment)
- `modify` copies `Impl` *iff* object is shared
- `modify` takes an *action*, ...

std::copy_on_write (P4210)

```
class Foo
{
public:
    Foo(std::size_t s) : _impl{std::in_place, s} {}

    bool operator==(Foo const& other) const = default;
    auto operator<=>(Foo const& other) const = default;

    bool empty() const {
        return _impl->size() != 0;
    }

    void resize(std::size_t s) {
        _impl.modify([&](auto& impl) { impl.resize(s); });
    }

    void push_front(int val) {
        _impl.modify(
            [&](std::vector<int>& impl) {
                impl.push_front(val);
            },
            [&](std::vector<int> const& impl) {
```

- Copy has $O(1)$ complexity (atomic increment)
- `modify` copies `Impl` *iff* object is shared
- `modify` takes an *action*, ...
- ... and (optionally) a *transformation*.

Guidelines

- Wrap all pointers in custom classes with regular semantics.

Guidelines (pre C++26)

- Use `std::shared_ptr<T const>` for immutable types when copy is more expensive than atomic increment.
- Otherwise, use `std::weak_ptr<T>` .

Guidelines (C++26)

- Use `std::shared_ptr<T const>` for immutable types when copy is more expensive than atomic increment.
- Otherwise, use `std::indirect<T>` for things that you allocate.
- Use `std::unique_ptr<T>` with a custom deleter for wrapping C APIs.

Guidelines (Future)

- Use `std::copy_on_write<T>` when copy is more common than modifications and copy is more expensive than atomic increment.

And now for something completely different.

Runtime Polymorphism

```
class IWidget
{
public:
    virtual void foo() = 0;
};
```

C++ Core Guideline

C.35: A base class destructor should be either public and virtual, or protected and non-virtual

Reason To prevent undefined behavior. If the destructor is public, then calling code can attempt to destroy a derived class object through a base class pointer, and the result is undefined if the base class's destructor is non-virtual. If the destructor is protected, then calling code cannot destroy through a base class pointer and the destructor does not need to be virtual; it does need to be protected, not private, so that derived destructors can invoke it. In general, the writer of a base class does not know the appropriate action to be done upon destruction.

Runtime Polymorphism

```
class IWidget
{
public:
    virtual void foo() = 0;
};
```

Runtime Polymorphism

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    virtual void foo() = 0;
};
```

C++ Core Guideline

C.67: A polymorphic class should suppress public copy/move

Reason A *polymorphic class* is a class that defines or inherits at least one virtual function. It is likely that it will be used as a base class for other derived classes with polymorphic behavior. If it is accidentally passed by value, with the implicitly generated copy constructor and assignment, we risk slicing: only the base portion of a derived object will be copied, and the polymorphic behavior will be corrupted.

If the class has no data, `=delete` the copy/move functions. Otherwise, make them protected.

Runtime Polymorphism

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    virtual void foo() = 0;
};
```

Runtime Polymorphism

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    IWidget(IWidget const&) = delete;
    IWidget& operator=(IWidget const&) = delete;

    virtual void foo() = 0;
};
```

Runtime Polymorphism

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    IWidget(IWidget const&) = delete;
    IWidget& operator=(IWidget const&) = delete;

    void foo() {
        this->xfoo();
    }

private:
    virtual void xfoo() = 0;
};
```

Non-Virtual Interface (NVI):

separate the specification of interface from the specification of the implementation's customizable behavior



Closed

rule proposal: C.141 Prefer to make virtual functions private. #768



hsutter on Oct 15, 2016

Contributor



FWIW, I find NVI sound but I don't want it to be a distraction so perhaps we should remove it for that reason.

Slightly longer:

Pro: It's been rediscovered multiple times in multiple languages, and last time I looked the standard library de facto follows it closely -- except mainly for `std::exception::what()`, when I looked pretty much every other virtual function in `std::` was nonpublic.

Con: NVI can be abused (e.g., duplicating public and virtual functions) and can definitely be a distraction here. I'm also fine with removing NVI for the reason of its being a distraction, just so we can spend the time on lots of other good stuff instead.



gdr-at-ms on Oct 15, 2016

Last edited by gdr-at-ms ▾

Contributor



If you have a good module system where private members aren't exported, it becomes most confusing to have to override something you can't even see, let alone access.



BjarneStroustrup on Oct 17, 2016

Contributor



I do not think we should encourage NVI. Often, the non-virtual function does nothing but forward, doubling the number of member functions and requiring a fancy naming scheme. Like a trivial getter or setter it prepares for a change that never happens.



Runtime Polymorphism

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    IWidget(IWidget const&) = delete;
    IWidget& operator=(IWidget const&) = delete;

    void foo() {
        this->xfoo();
    }

private:
    virtual void xfoo() = 0;
};
```

Runtime Polymorphism

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    IWidget(IWidget const&) = delete;
    IWidget& operator=(IWidget const&) = delete;

    virtual void foo() = 0;
};
```

Runtime Polymorphism

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    IWidget(IWidget const&) = delete;
    IWidget& operator=(IWidget const&) = delete;

    virtual void foo() const = 0;
};
```

Should `virtual` functions be `const` ?

Runtime Polymorphism

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    IWidget(IWidget const&) = delete;
    IWidget& operator=(IWidget const&) = delete;

    virtual void foo() const = 0;
};
```

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    IWidget(IWidget const&) = delete;
    IWidget& operator=(IWidget const&) = delete;

    virtual void foo() = 0;
};
```

Runtime Polymorphism

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    IWidget(IWidget const&) = delete;
    IWidget& operator=(IWidget const&) = delete;

    virtual void foo() const = 0;
};

class Widget
{
public:
    template <std::derived_from<IWidget> T>
    Widget(T value)
        : _impl{std::make_shared<T>(std::move(value))}
    {}

    void foo() const { _impl->foo(); }

private:
```

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    IWidget(IWidget const&) = delete;
    IWidget& operator=(IWidget const&) = delete;

    virtual void foo() = 0;
};
```

Runtime Polymorphism

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    IWidget(IWidget const&) = delete;
    IWidget& operator=(IWidget const&) = delete;

    virtual void foo() const = 0;
};

class Widget
{
public:
    template <std::derived_from<IWidget> T>
    Widget(T value)
        : _impl{std::make_shared<T>(std::move(value))}
    {}

    void foo() const { _impl->foo(); }

private:
```

Sean Parent: "I am not convinced that polymorphism ever requires mutability".

Runtime Polymorphism

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    IWidget(IWidget const&) = delete;
    IWidget& operator=(IWidget const&) = delete;

    virtual void foo() const = 0;
};

class Widget
{
public:
    template <std::derived_from<IWidget> T>
    Widget(T value)
        : _impl{std::make_shared<T>(std::move(value))}
    {}

    void foo() const { _impl->foo(); }

private:
```

But: Why decide in a base class whether a derived class may modify its data members?

Runtime Polymorphism

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    IWidget(IWidget const&) = delete;
    IWidget& operator=(IWidget const&) = delete;

    virtual void foo() const = 0;
};

class Widget
{
public:
    template <std::derived_from<IWidget> T>
    Widget(T value)
        : _impl{std::make_shared<T>(std::move(value))}
    {}

    void foo() const { _impl->foo(); }

private:
```

We should be able to archive independence with copying!

Runtime Polymorphism (C++26)

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    IWidget(IWidget const&) = delete;
    IWidget& operator=(IWidget const&) = delete;

    virtual void foo() const = 0;
};

class Widget
{
public:
    template <std::derived_from<IWidget> T>
    Widget(T value)
        : _impl{std::make_shared<T>(std::move(value))}
    {}

    void foo() const { _impl->foo(); }

private:
```

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    IWidget(IWidget const&) = delete;
    IWidget& operator=(IWidget const&) = delete;

    virtual void foo() = 0;
};
```

Runtime Polymorphism (C++26)

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    IWidget(IWidget const&) = delete;
    IWidget& operator=(IWidget const&) = delete;

    virtual void foo() const = 0;
};

class Widget
{
public:
    template <std::derived_from<IWidget> T>
    Widget(T value)
        : _impl{std::make_shared<T>(std::move(value))}
    {}

    void foo() const { _impl->foo(); }

private:
```

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    virtual void foo() = 0;
};
```

Runtime Polymorphism (C++26)

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    IWidget(IWidget const&) = delete;
    IWidget& operator=(IWidget const&) = delete;

    virtual void foo() const = 0;
};

class Widget
{
public:
    template <std::derived_from<IWidget> T>
    Widget(T value)
        : _impl{std::make_shared<T>(std::move(value))}
    {}

    void foo() const { _impl->foo(); }

private:
```

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    virtual void foo() = 0;
};

class Widget
{
public:
    template <std::derived_from<IWidget> T>
    Widget(T value)
        : _impl{std::move(value)}
    {}

    void foo() { _impl->foo(); }

private:
    std::polymorphic<IWidget> _impl;
};
```

Guidelines

Guidelines

- Wrap all pointers in custom classes with regular semantics.

Guidelines

- Wrap all pointers in custom classes with regular semantics.
- For polymorphic types,
 - implement in terms of `std::shared_ptr<Base const>` when all `virtual` functions are `const` ,
 - otherwise, use `std::polymorphic<Base>` .

Guidelines

- Wrap all pointers in custom classes with regular semantics.
- For polymorphic types,
 - implement in terms of `std::shared_ptr<Base const>` when all `virtual` functions are `const` ,
 - otherwise, use `std::polymorphic<Base>` .
- I call this the NVI 2.0.
 - The specification of the interface and the specification of the implementation's customizable behavior are in two distinct classes.

Neither `std::shared_ptr<Base const>` nor `std::polymorphic<Base>`
destroys the object through `Base` pointer!

A base class destructor should be either ~~public and virtual~~, or protected and non-virtual.

Runtime Polymorphism (C++26)

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    IWidget(IWidget const&) = delete;
    IWidget& operator=(IWidget const&) = delete;

    virtual void foo() const = 0;
};

class Widget
{
public:
    template <std::derived_from<IWidget> T>
    Widget(T value)
        : _impl{std::make_shared<T>(std::move(value))}
    {}

    void foo() const { _impl->foo(); }

private:
```

```
class IWidget
{
public:
    virtual ~IWidget() = default;

    virtual void foo() = 0;
};

class Widget
{
public:
    template <std::derived_from<IWidget> T>
    Widget(T value)
        : _impl{std::move(value)}
    {}

    void foo() { _impl->foo(); }

private:
    std::polymorphic<IWidget> _impl;
};
```

Runtime Polymorphism (C++26)

```
class IWidget
{
protected:
    ~IWidget() = default;

public:
    IWidget(IWidget const&) = delete;
    IWidget& operator=(IWidget const&) = delete;

    virtual void foo() const = 0;
};

class Widget
{
public:
    template <std::derived_from<IWidget> T>
    Widget(T value)
        : _impl{std::make_shared<T>(std::move(value))}
    {}

    void foo() const { _impl->foo(); }
```

```
class IWidget
{
protected:
    ~IWidget() = default;

public:
    virtual void foo() = 0;
};

class Widget
{
public:
    template <std::derived_from<IWidget> T>
    Widget(T value)
        : _impl{std::move(value)}
    {}

    void foo() { _impl->foo(); }

private:
    std::polymorphic<IWidget> _impl;
};
```

Why inheritance and `virtual` functions at all?

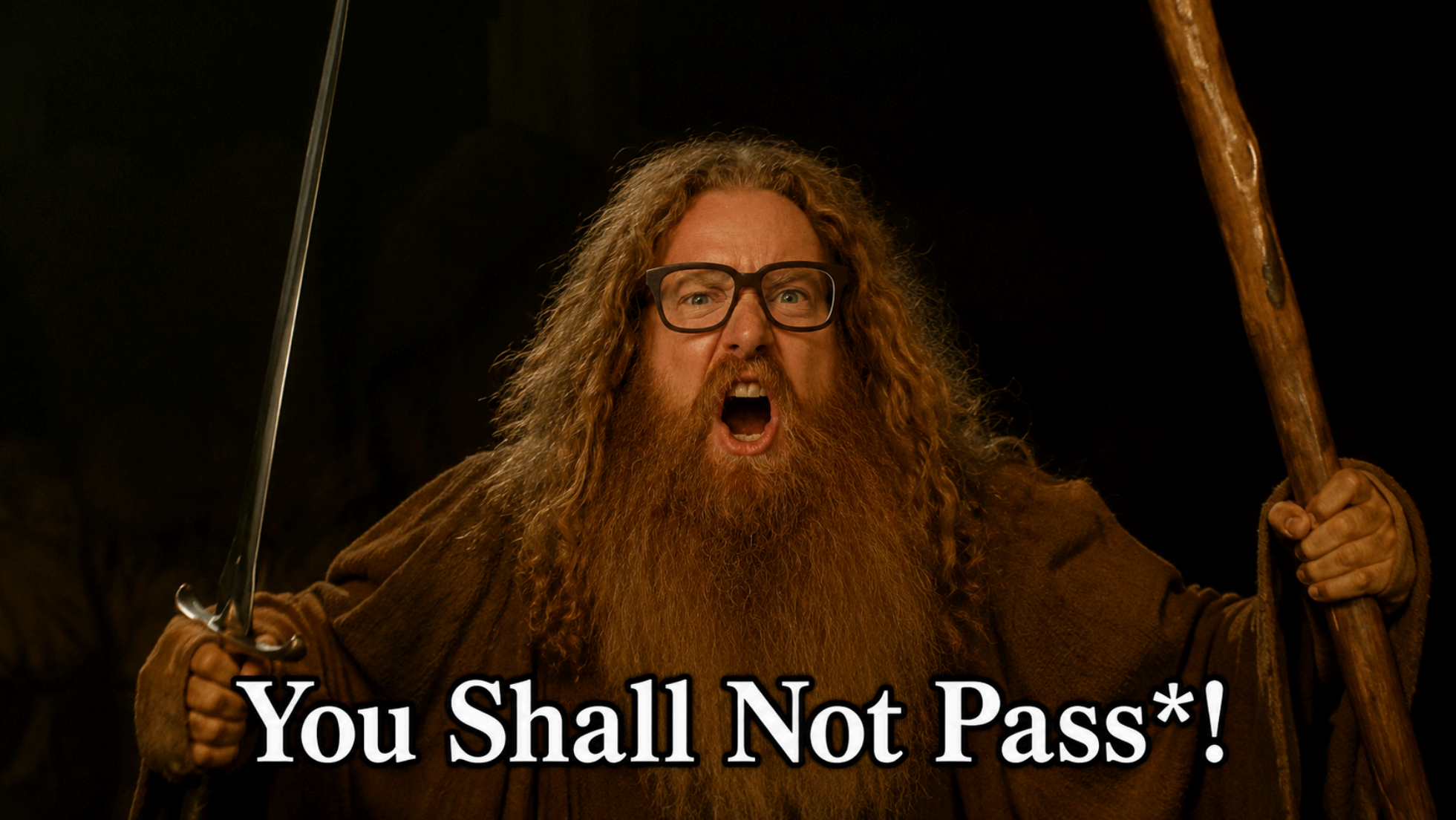
```
struct Container
{
    auto size() const → std::size_t;
    auto empty() const → bool;
    auto clear() → void;
};

rjk::duck<Container> d{std::vector{1, 2, 3}};
d.size(); // 3
d = std::string{"hello"}; // swap type at runtime
d.size(); // 5
d = std::map<int, int>{{1, 2}, {3, 4}};
d.empty(); // false
d.clear();
d.empty(); // true
```

`std::vector`, `std::string`, and `std::map` are structural subtypes of `Container` according to LSP.

Summary

- Pointers - smart or not - shall not appear in function signatures.
- Wrap pointer in custom classes with regular semantics.
 - `std::shared_ptr` and `std::weak_ptr` are useful building blocks.
 - `std::indirect` and `std::polymorphic` are better building blocks.
 - More building blocks are needed.

A man with a long, curly brown beard and glasses, wearing a brown robe, is shown from the chest up. He is holding a sword in his right hand and a wooden staff in his left hand. His mouth is wide open in a shout. The background is dark and indistinct.

You Shall Not Pass*!